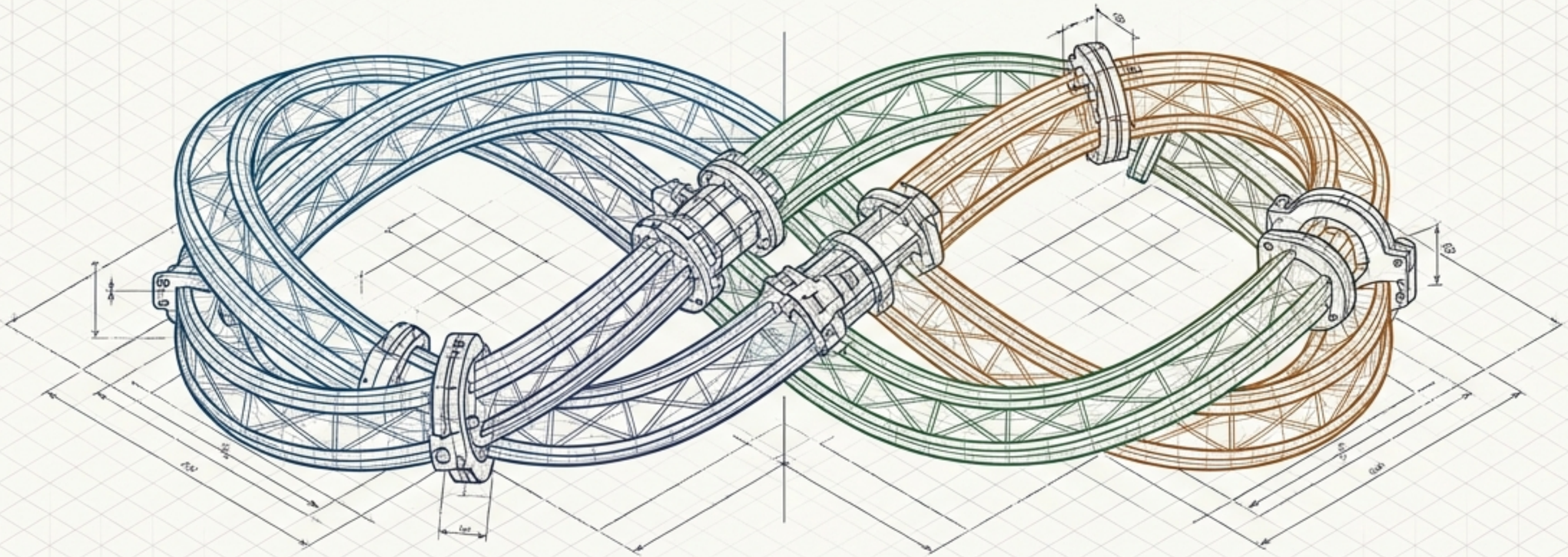
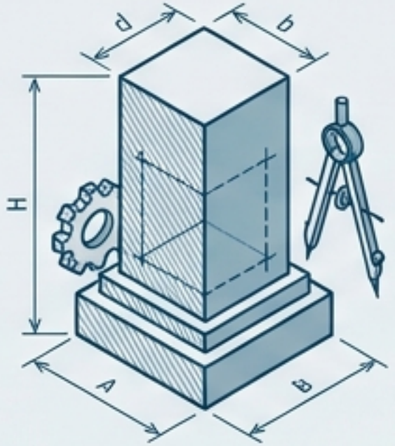
THE MODERN SOFTWARE ENGINEERING BLUEPRINT



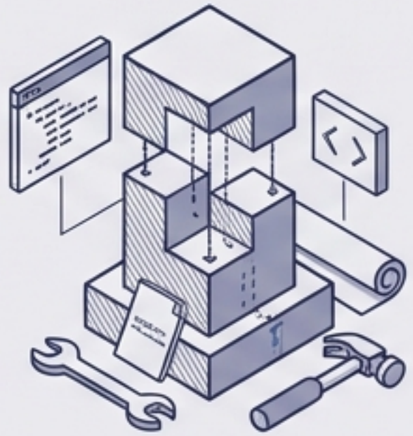
A structural guide to the technical, collaborative, and managerial systems that drive the software development life cycle.

The Four Pillars of the Software Process



Specification

Understanding and defining required services and operational constraints. Mistakes here inevitably compound later.

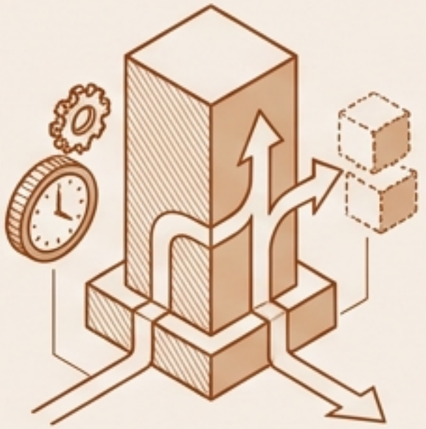


Development

The process of designing and implementing the executable system.

Validation

Ensuring the system conforms to its specification and meets customer expectations.



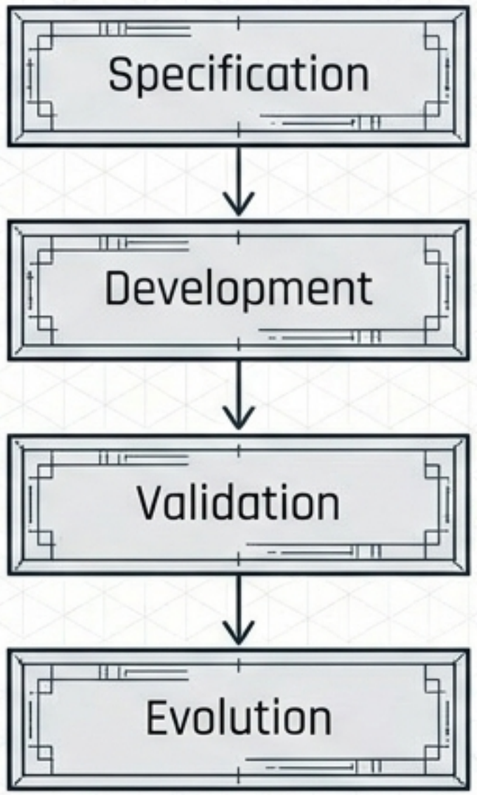
Evolution

Continually modifying the software over its lifetime to meet changing needs.

Real software processes are interleaved sequences of these four activities—not isolated steps.

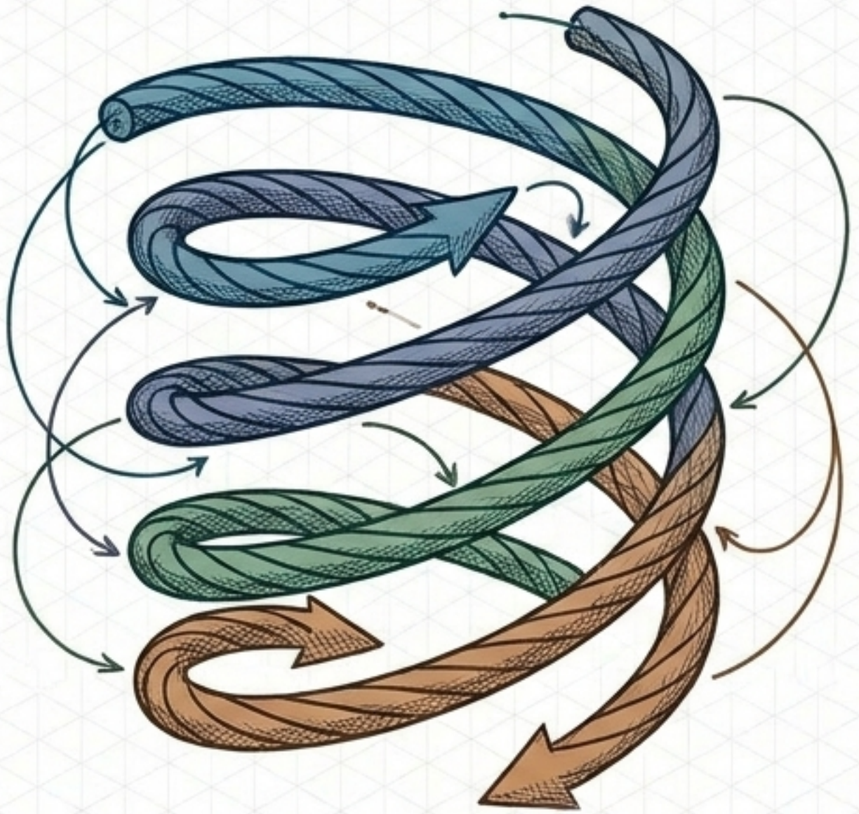
Breaking the Linear Illusion

The Plan-Driven Waterfall



Organizes activities strictly in sequence. Specification must finish before Development begins.

The Agile / Incremental Reality

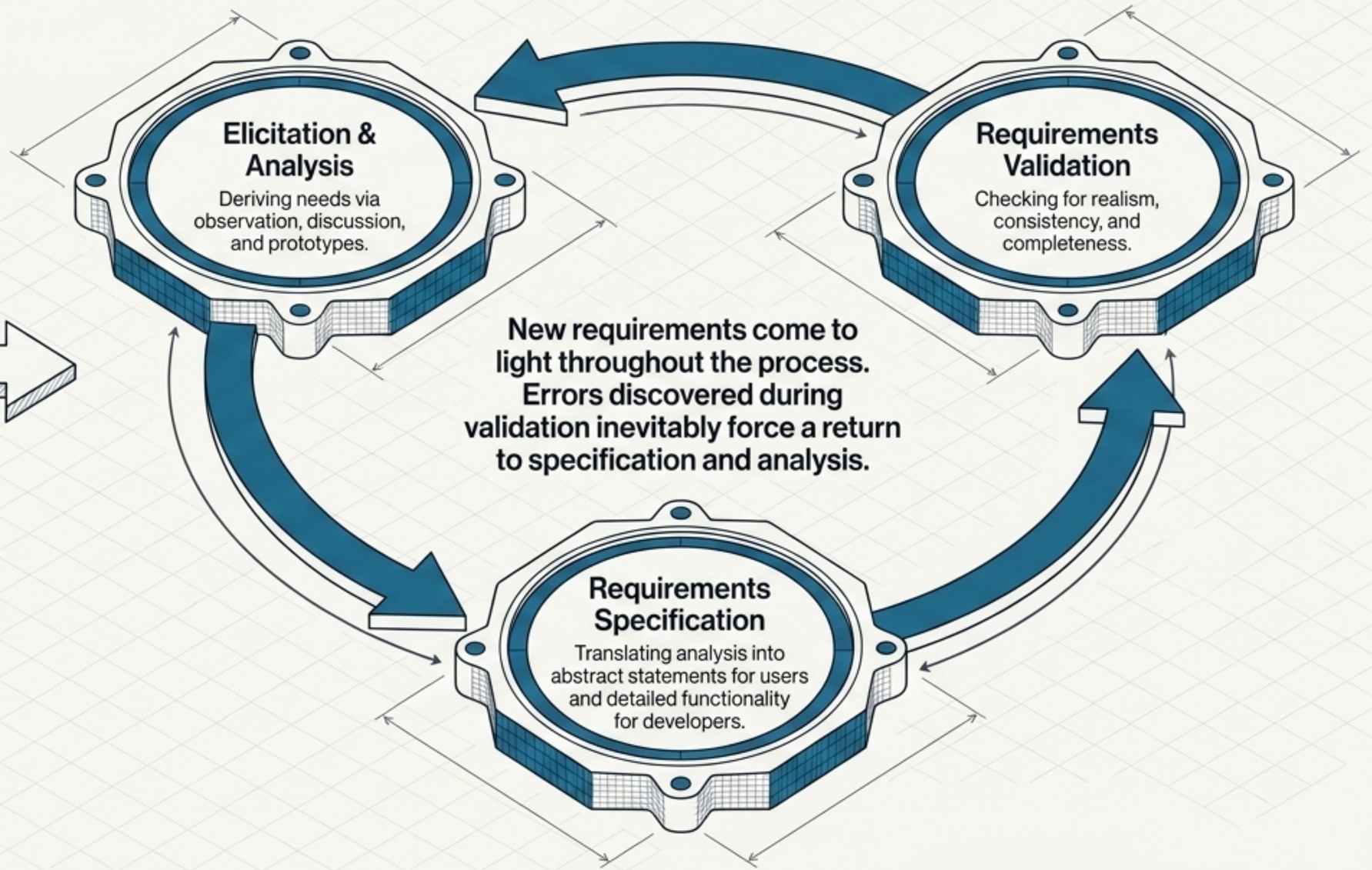


In modern incremental development, activities are intensely interleaved. New information constantly forces backtracking, and requirements are defined just before a system increment is built.

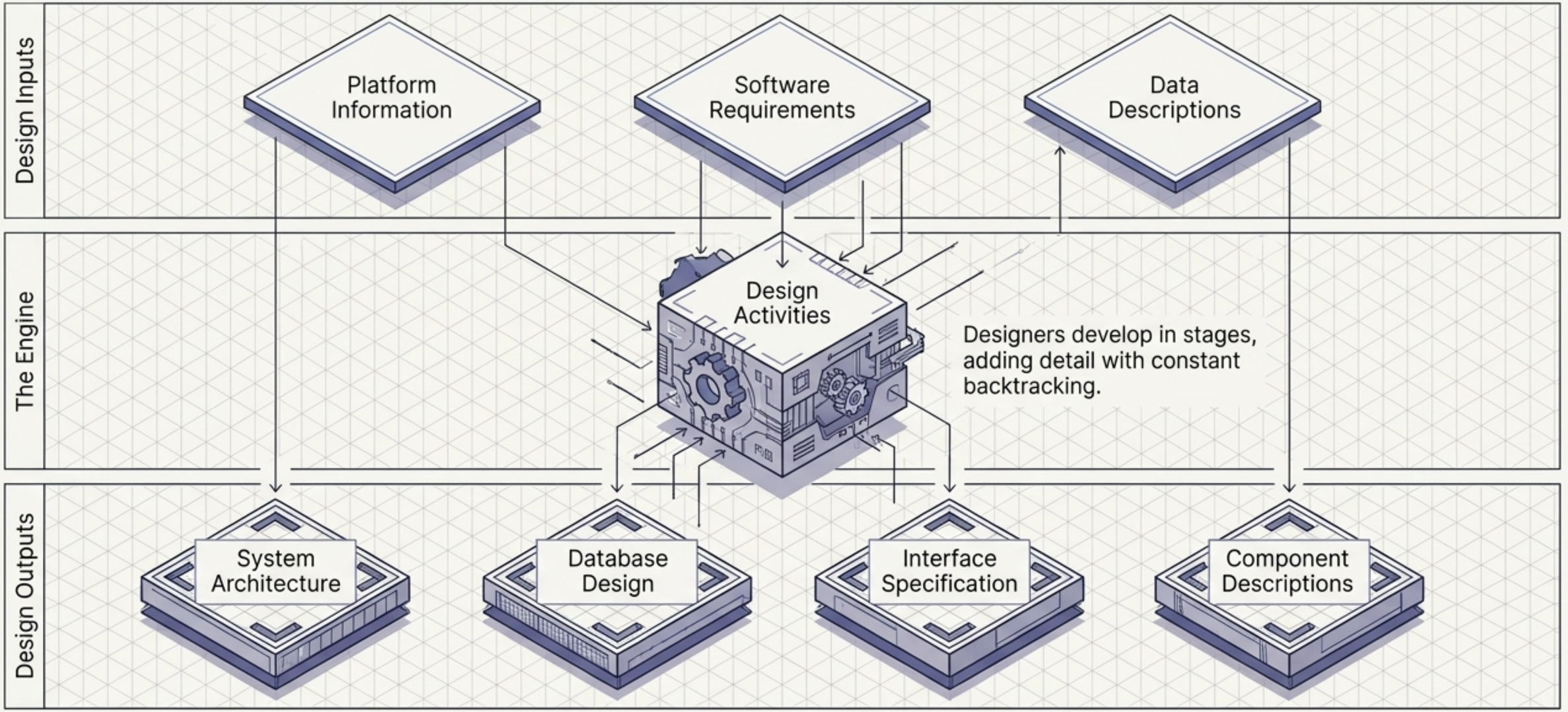
Phase 1: The Requirements Engineering Loop

Pre-Loop: Feasibility Studies

Short-term, relatively cheap marketing/technical studies conducted before the loop begins to decide whether to proceed.

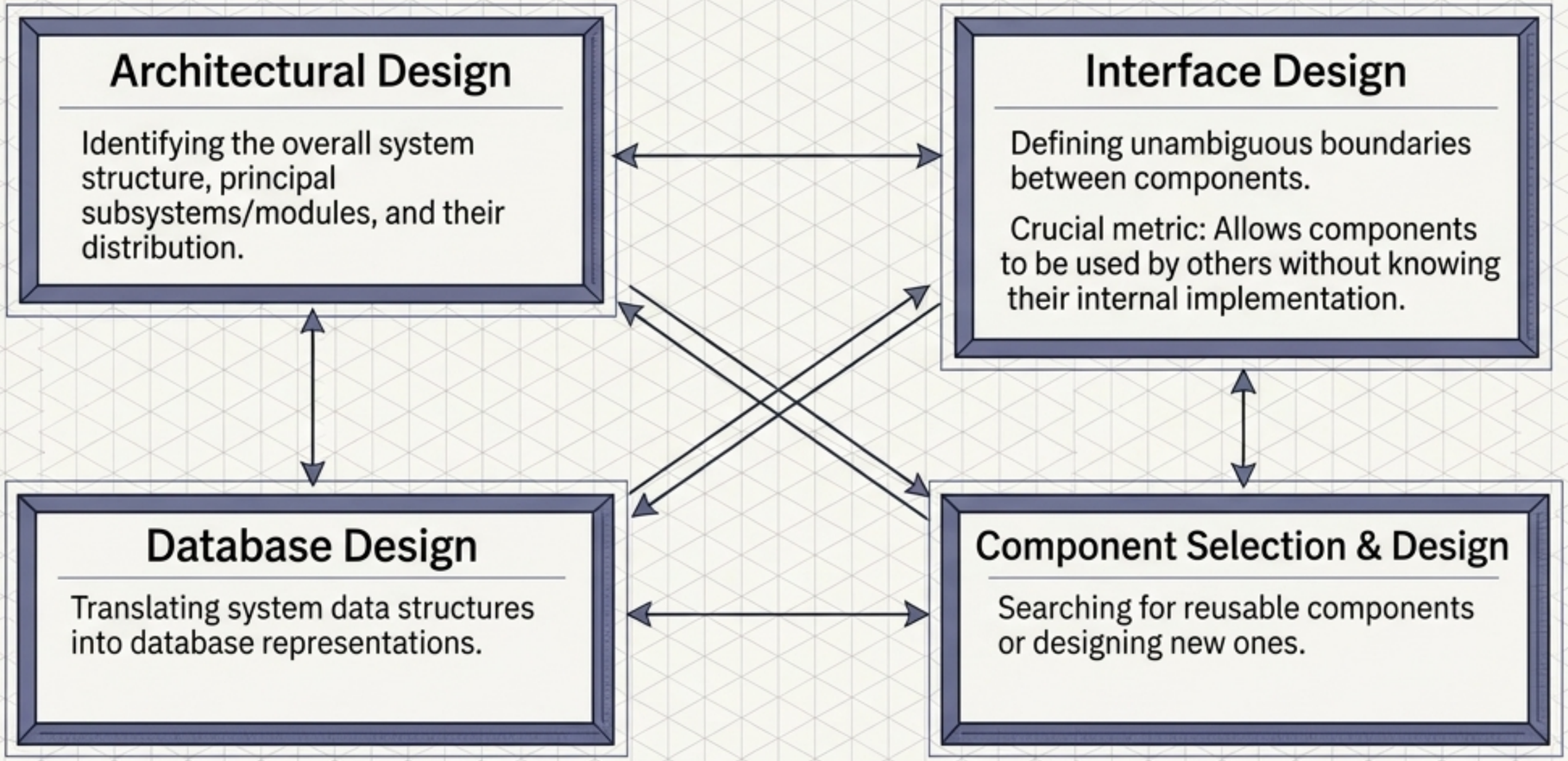


Phase 2: The Design and Implementation Engine



In model-driven approaches, outputs are formal diagrams. In agile, outputs may simply be represented directly in the program code.

The Four Interdependent Dimensions of Design



New information is constantly generated, rendering design rework inevitable.

Translating Design into Execution

The Art of Programming

- Programming is an individual activity with no universal process.
- Design and implementation are deeply interleaved.
- Software tools often generate a “skeleton program”, leaving the programmer to add operational details.

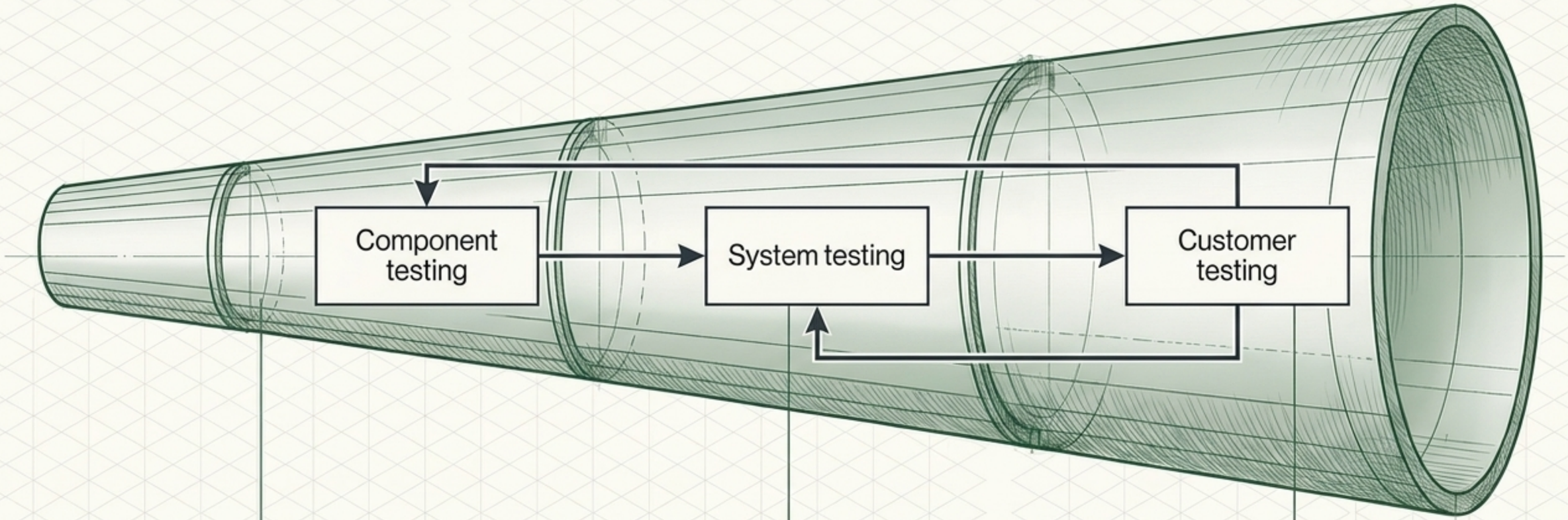
Testing vs. Debugging

Defect Testing 	Establishes the existence of defects.
Debugging 	Locates and corrects those defects.

The Debugging Loop

Generate hypotheses about output anomalies → Trace program code → Deploy interactive tools

Phase 3: The Expanding Scope of System Validation



Stage 1: Component Testing (Micro)

Independent testing of functions or object classes. Usually done by the developers themselves.

Tooling: Automated test runners.

Stage 2: System Testing (Macro)

Integrating components to find unanticipated interactions and interface problems.

Validates emergent properties.

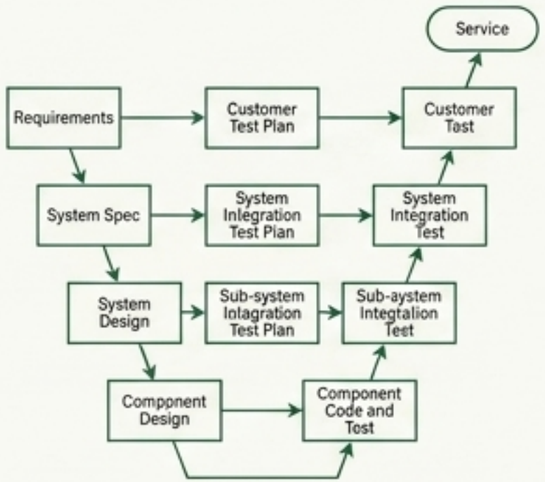
Stage 3: Customer Testing (Real World)

Validating against real customer data.

Crucial for revealing requirements omissions, as real data exercises the system differently than simulated test data.

Validation Frameworks in Practice

The V-Model (Plan-Driven)

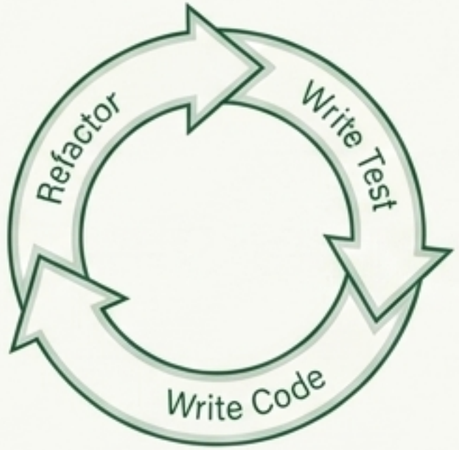


Mechanism: Testing is driven by independent test plans derived directly from system specification and design stages.

Execution: Independent testing teams execute tests.

Use Case: Critical systems development requiring strict traceability.

Test-Driven Development (Agile)



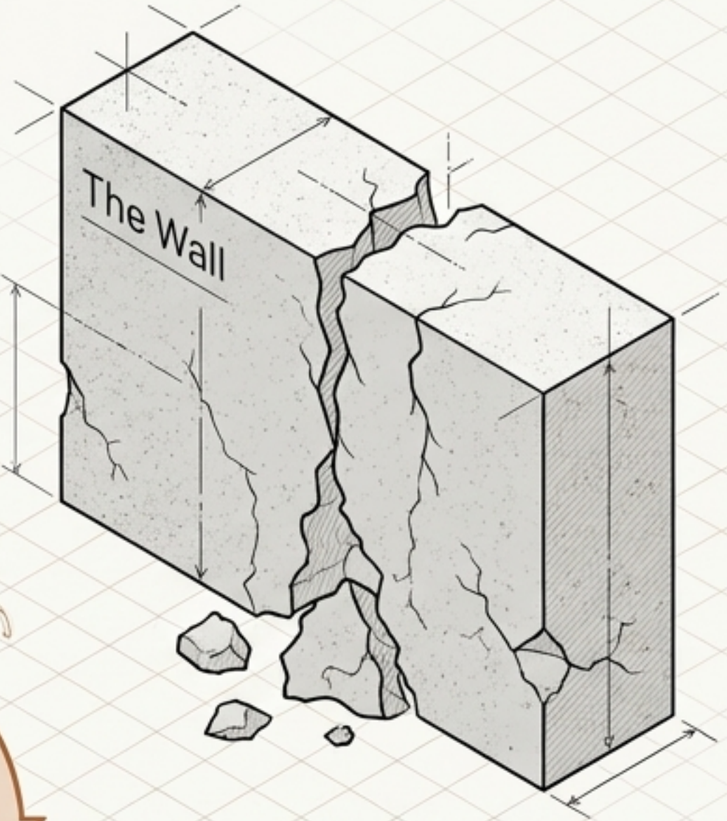
Mechanism: Tests are developed alongside requirements before implementation begins.

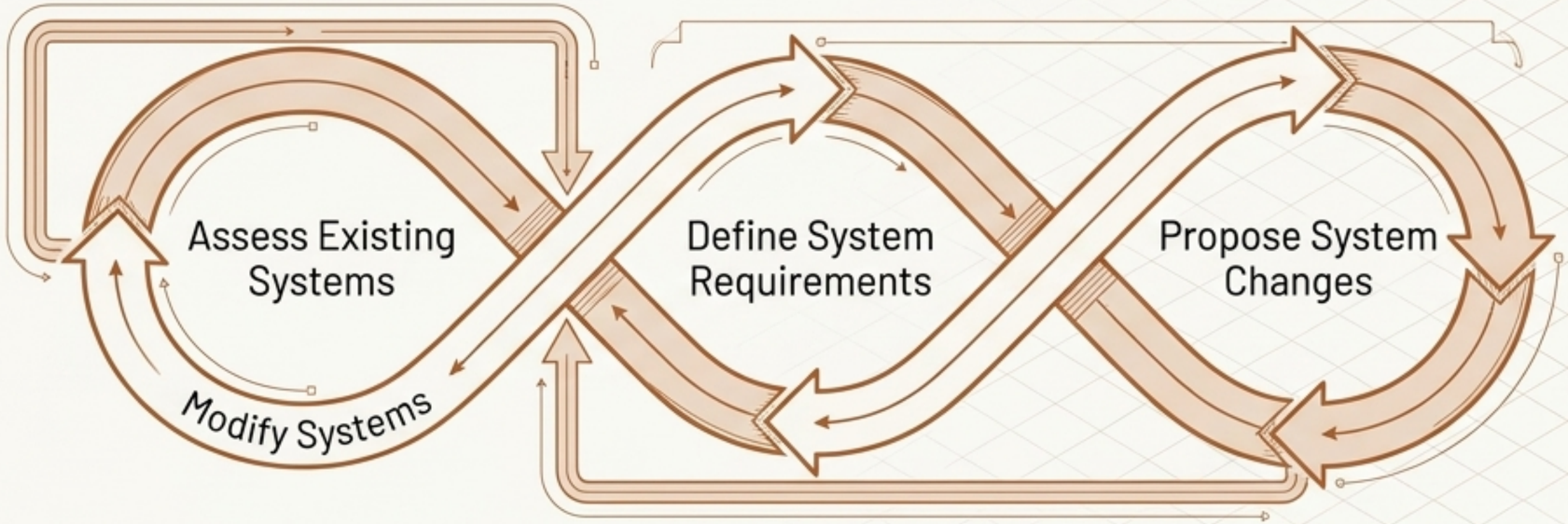
Execution: Programmers write tests incrementally.

Use Case: Normal agile processes to ensure immediate understanding of requirements without delays.

Shattering the Development /Maintenance Divide

Historically, software engineering split original development from maintenance. This distinction is increasingly irrelevant. Very few systems are completely new.

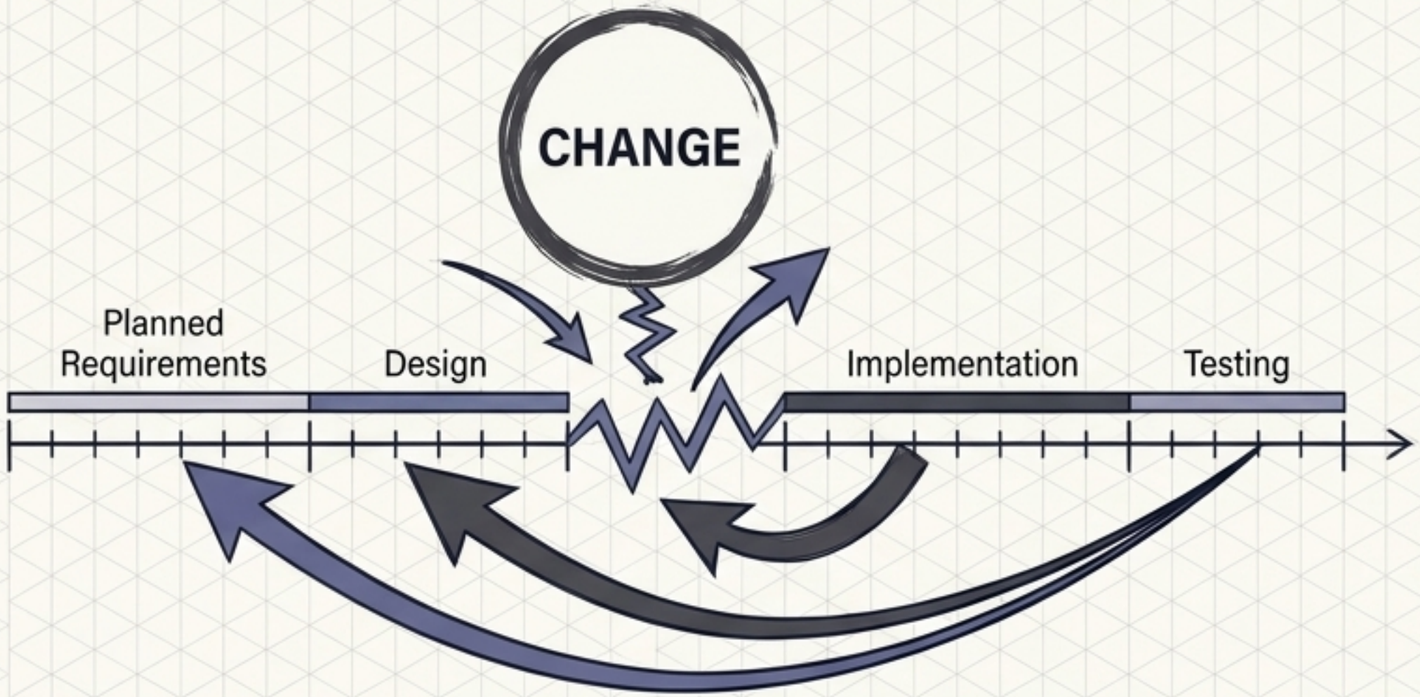




Software engineering is not a one-off build; it is an evolutionary process continually responding to changing customer needs.

The Inevitability of Rework

Change is inevitable in all large software projects due to external pressures, competition, shifting management priorities, and new technologies.



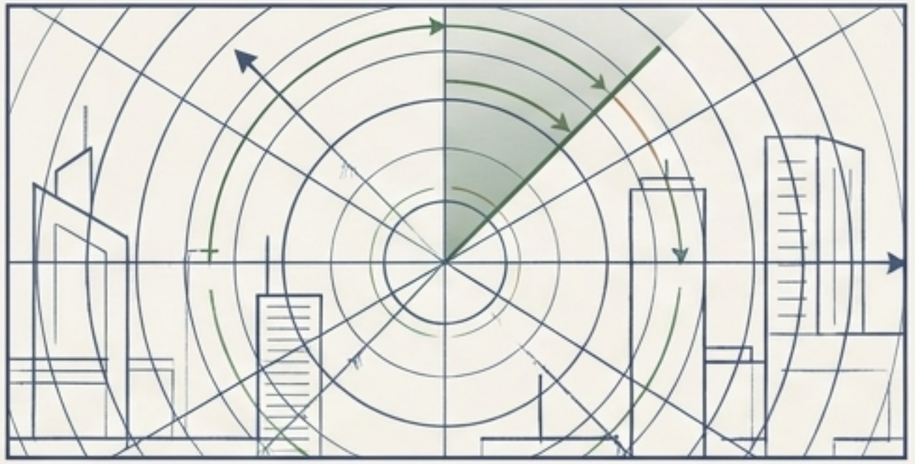
The Cost Function

1. New requirements are identified mid-stream.
2. Previous requirements analysis must be repeated.
3. System must be redesigned to accommodate.
4. Implemented code is altered and retested.

Rework — The added cost of software development where completed work must be redone to accommodate systemic changes.

Strategic Frameworks for Coping with Change

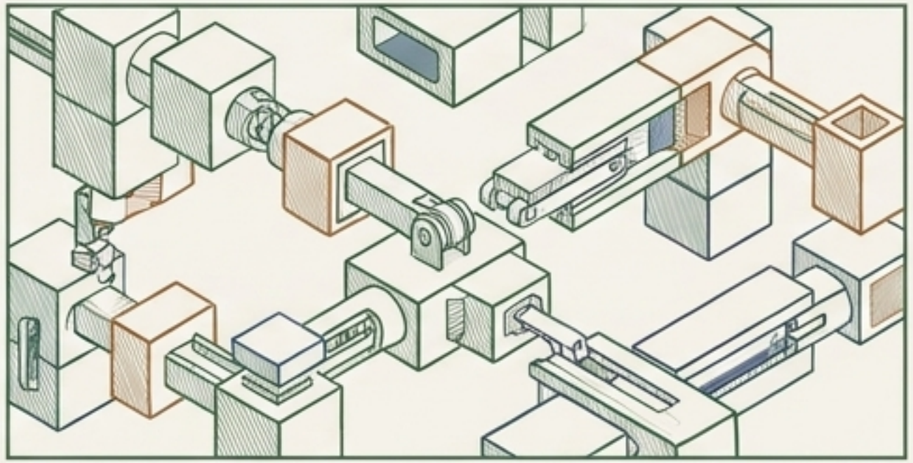
Path A: Change Anticipation



Philosophy:
Predict possible changes before significant rework is required.

Execution:
Build prototype systems. Allow customers to experiment and refine requirements prior to committing to high production costs.

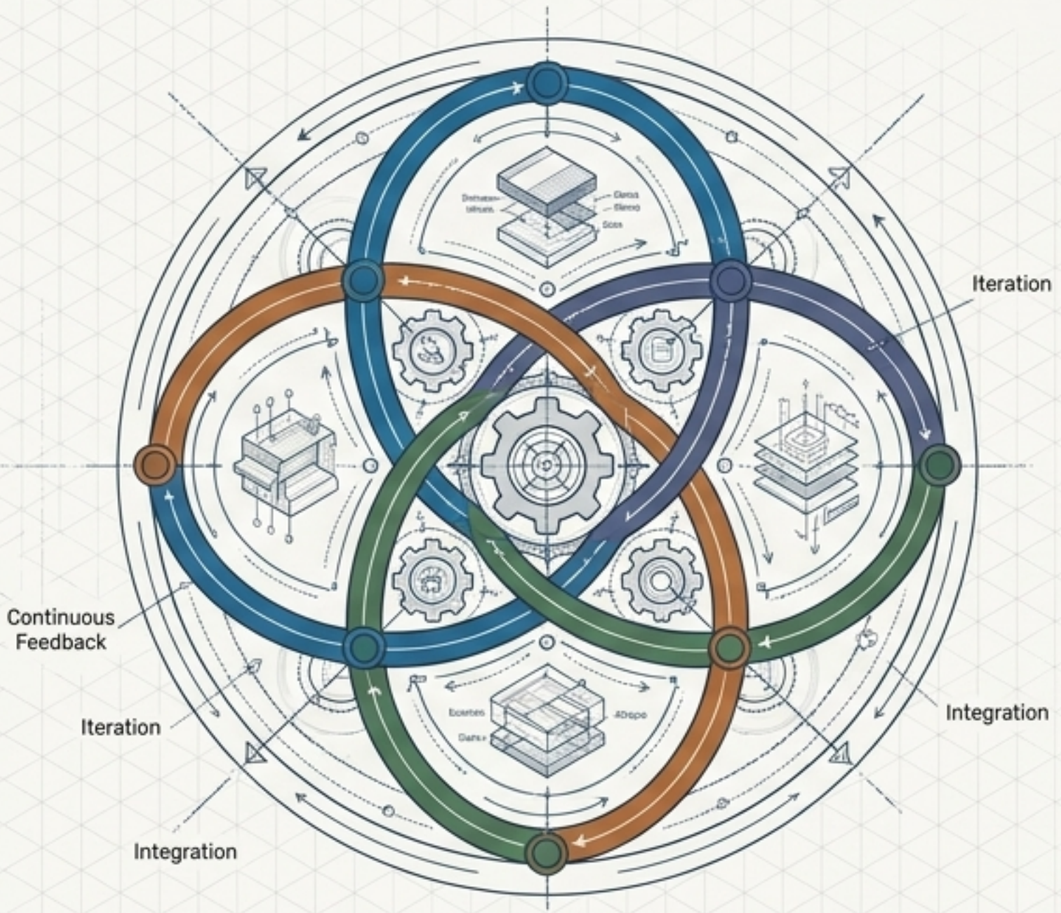
Path B: Change Tolerance



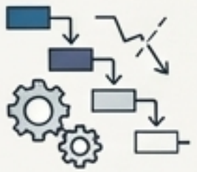
Philosophy:
Design the process and architecture so changes are easily absorbed.

Execution:
Utilize incremental development. Implement proposed changes in future, un-developed increments, or isolate alterations to a single, small existing increment.

Software as an Interleaved, Living System

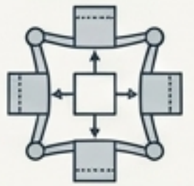


- The rigid sequences of the Waterfall model are illusions; real engineering relies on highly interleaved activities.



- Specification, Design, Validation, and Evolution are not isolated phases, but continuous feedback loops driven by new information.

- Success relies on designing systems and processes that either anticipate or seamlessly tolerate inevitable change.



Software development is not the manufacturing of a static product, but the cultivation of an evolutionary system.